

React Hook

React Hooks

Hooks are functions that let you "hook into" **React state** and lifecycle features from functional components

State Hooks
- useState
- useReducer

Context Hooks
- useContext

Ref Hooks
- useRef
- useImperativeHandle

Effect Hooks
- useEffect

Performance Hooks
- useMemo
- useCallBack

Other Hooks
- useActionState
- useId
- useDebugValue
- useSyncExternalStore

use….., set….

useState

useState Hook allows us to track **state** in a function component

State generally refers to data or properties that need to be tracking in an application

```
function HookState() {
    const [count, setCount] = useState(0);
    return (
        <div>
            <h2>Count: {count}</h2>
            <button onClick={() => setCount(count + 1)}>Increment</button>
            <button onClick={() => setCount(count - 1)}>Decrement</button>
            <button onClick={() => setCount(0)}>Reset</button>
        </div>
    );
}
export default HookState;
```

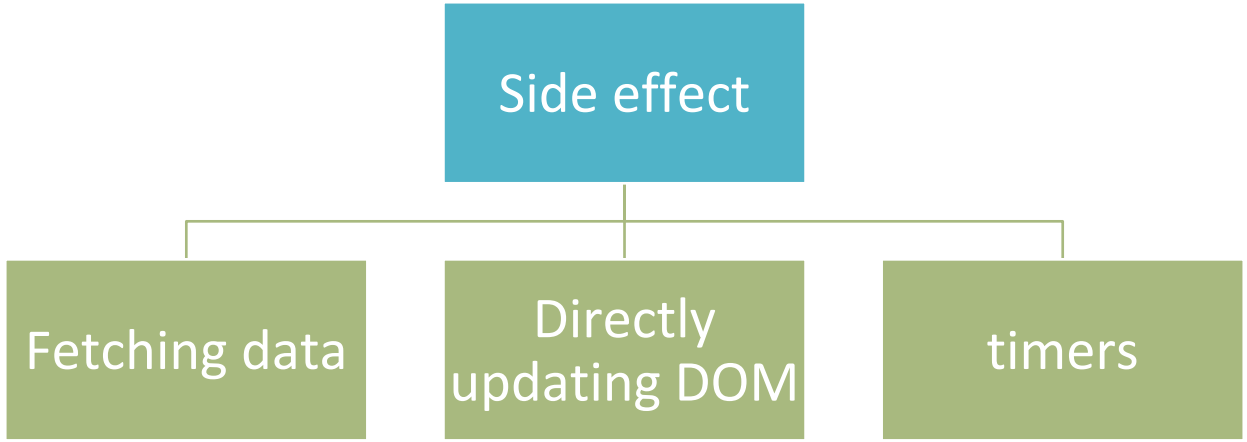


useEffect

The useEffect Hook used to handle side effects in functional components

Side effects refer to any operations that occur in a component after rendering

React will remember the function you passed (“effect”), and call it later after performing the DOM updates.



useEffect(<function>, <dependency>)

No dependency

```
useEffect(()=>{ ... }, [])
```

Render after first render only

With dependency

```
useEffect(()=>{... },
[dependency1, dependency2])
```

Render after first render AND subsequent renders caused by a **change** in a dependency

Render after first render only

useEffect(()=>{ ... }, [ ])

```
import { useState } from "react";
function HookTest() {
    const [count, setCount] = useState(0);

    setTimeout(() => {
        setCount(count + 1);
    }, 3000);

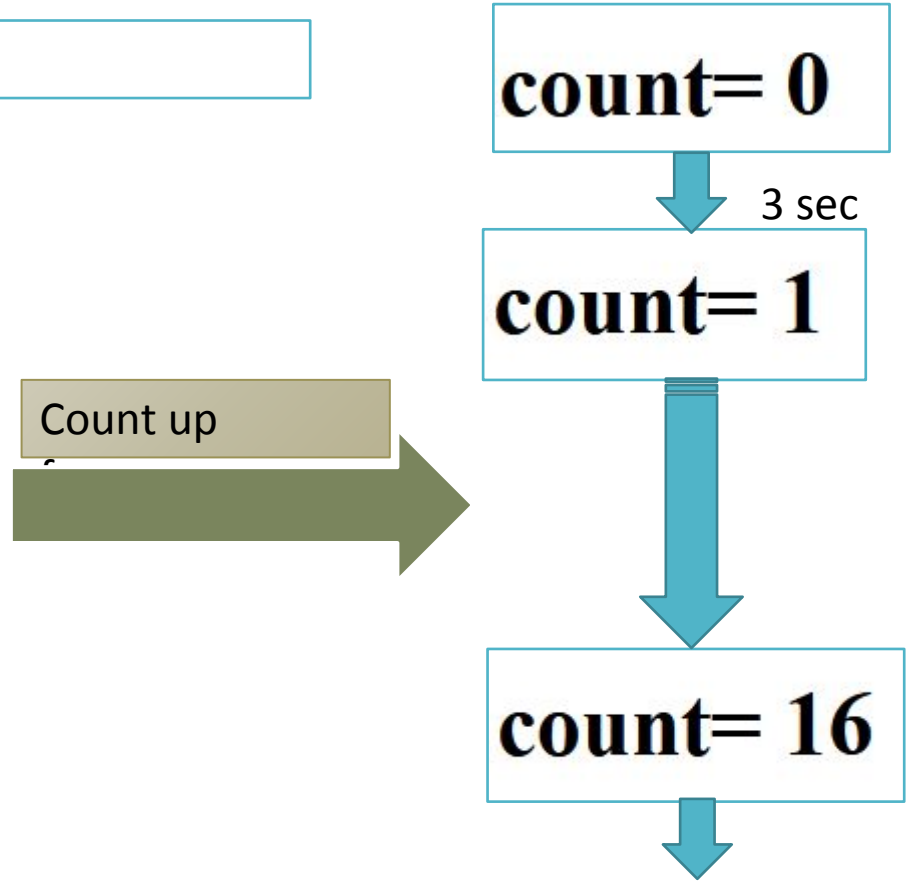
    return <h1>count= {count}</h1>;
}
export default HookTest;
```

Re-render

3 sec

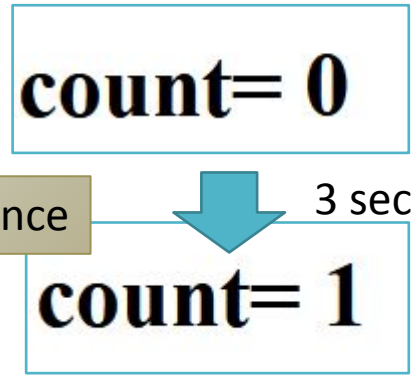
First render

Count up




```
useEffect(() => {
    setTimeout(() => {
        setCount(count + 1);
    }, 3000);
}, []);
```

Do once



Note: Some effects require cleanup to reduce memory leaks

```
useEffect(() => {
    let timer = setTimeout(() => {

    }, 1000);
    return () => clearTimeout(timer)
}, []);
```

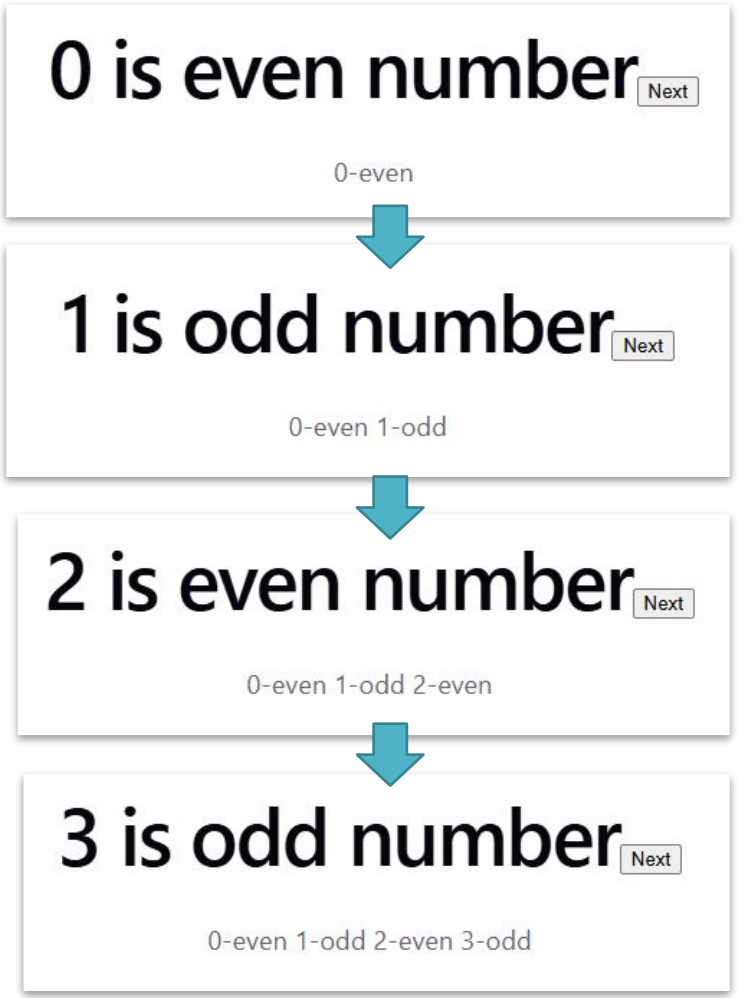
Render after first render AND subsequent renders caused by a change in a dependency

useEffect(()=>{... }, [dependency1, dependency2])

```
import { useState, useEffect } from "react";
function HookTest() {
    const [count, setCount] = useState(0);
    const [isCount, setIsCount] = useState(true);
    const [text, setText] = useState("");

    useEffect(() => {
        setIsEvent(count%2 == 0);
        setText(text+" "+count +"-"+ (count%2==0? "even":"odd"));
    }, [count]);

    return (<div><h1>{count} is {isEvent? "even":"odd"} number
<button onClick={()=>setCount(count+1)}Next</button></h1><p>{text}</p>
</div>);
}
export default HookTest;
```



multiple dependencies can included in the array => [count, name]

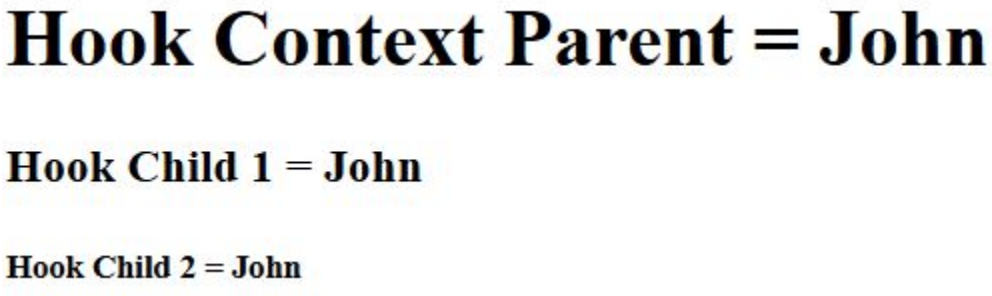
useContext

React Context is a way to manage state globally

```
function HookContext() {
    const[name, setName] = useState("John");
    return (
        <>
            <h1>Hook Context Parent = {name}</h1>
            <HookContextChild1 name={name}/>
        </>
    );
}
```

```
function HookContextChild1({name}) {
    return (
        <>
            <h3>Hook Child 1 = {name}</h3>
            <HookContextChild2 name={name}/>
        </>
    );
}
```

```
function HookContextChild2({name}) {
    return <h5>Hook Child 2 = {name}</h5>;
}
```



1. Create the Context

```
import { createContext } from 'react';

const UserContext = createContext();

export default UserContext;
```

2. Providing the context

```
import UserContext from "./UserContext";
import HookContextChild1 from "./HookContextChild1";

function HookContext() {
    const userName = "John";
    return (
        <UserContext.Provider value={userName}>
            <h1>Hook Context Parent = {userName}</h1>
            <HookContextChild1 />
        </UserContext.Provider>
    );
}
```

Set context value global

Hook Context Parent = John

Hook Child 1

Hook Child 2 = John

Creating a context,
wrapping everything in a provider,
using the useContext() in every consumer

However its increases complexity!!

```
import HookContextChild2 from "./HookContextChild2";
function HookContextChild1() {
    return (
        <>
            <h3>Hook Child 1 </h3>
            <HookContextChild2 />
        </>
    );
}
```

No context

3. Consuming the context

```
import { useContext } from "react";
import UserContext from "./UserContext";

function HookContextChild2() {
    const userName = useContext(UserContext);
    return <h5>Hook Child 2 = {userName}</h5>;
}
```



Import context



Get value

useContext - multiple values

```
import UserContext from "./UserContext";
import { useState } from "react";
import HookContextChild1 from "./HookContextChild1";

function HookContext() {
    const userName = "John";
    const [isAdmin, setIsAdmin] = useState(true);
    const contextValue = {
        userName,
        isAdmin,
        setIsAdmin,
    };

    return (
        <UserContext.Provider value={contextValue}>
            <h1> User = {userName} is {!isAdmin ? "not" : ""} Admin</h1>
            <HookContextChild1 />
        </UserContext.Provider>
    );
}
```

Set context value global

```
function HookContextChild2() {
    const { userName, isAdmin, setIsAdmin } = useContext(UserContext);
    return (<>
        <h5>Hook Child 2 = {userName} is {!isAdmin ? "not" : ""} Admin</h5>
        <button onClick={() => setIsAdmin(!isAdmin)}>Toggle Admin</button>
    </>);
}
```

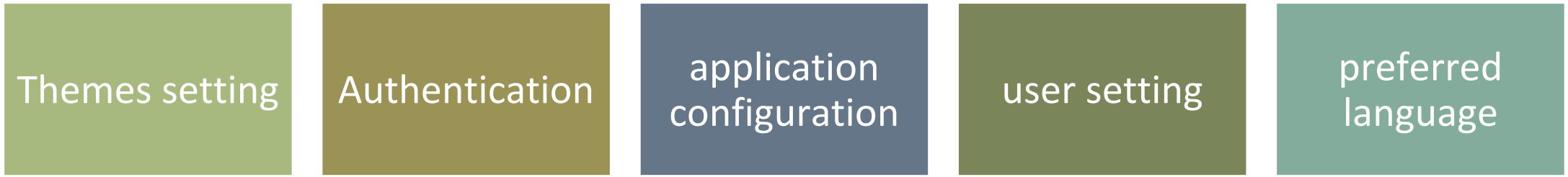


useContext - When to Use

need **global state** management.

We want to avoid prop drilling.

We need state sharing between multiple components without a third-party state management library



useRef

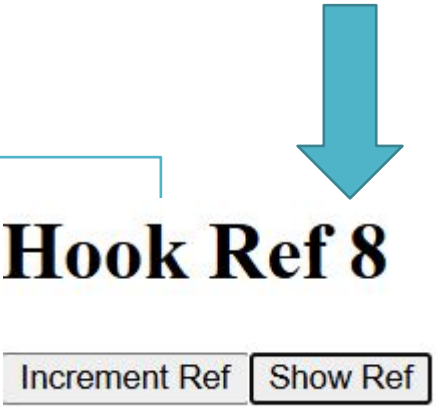
Increment value but no change



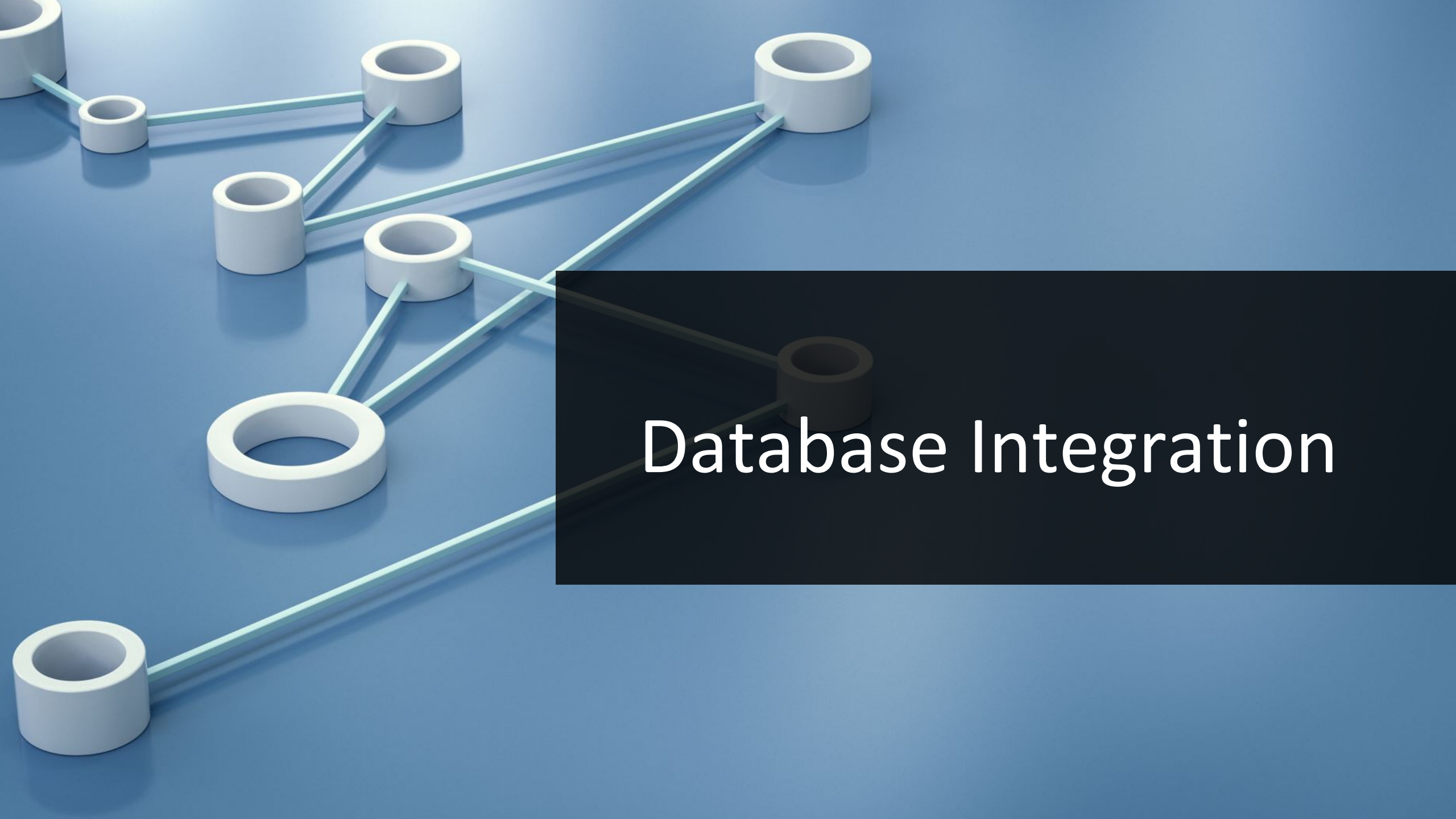
The useRef Hook allows you to persist values between renders

```
import { useRef } from "react";
import { useState } from "react";
function HookRef() {
    const countRef = useRef(0);
    const [trigger, setTrigger] = useState(true);

    return (<>
        <h1>Hook Ref {countRef.current}</h1>
        <button onClick={() => countRef.current++}>Increment Ref</button>
        <button onClick={() => setTrigger(!trigger)}>Show Ref</button>
    </>);
}
```



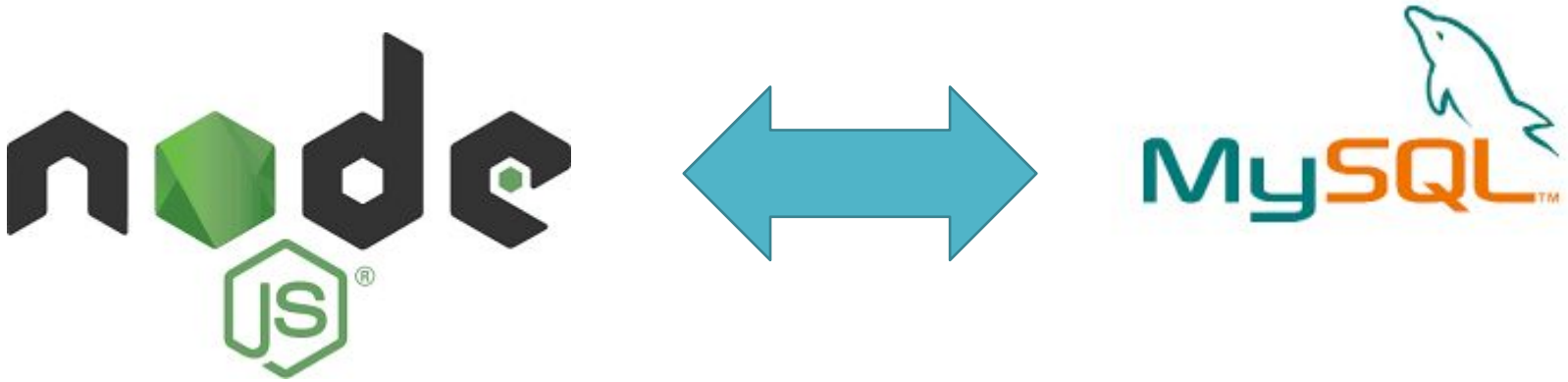
Change value for re-render



Database Integration

installation

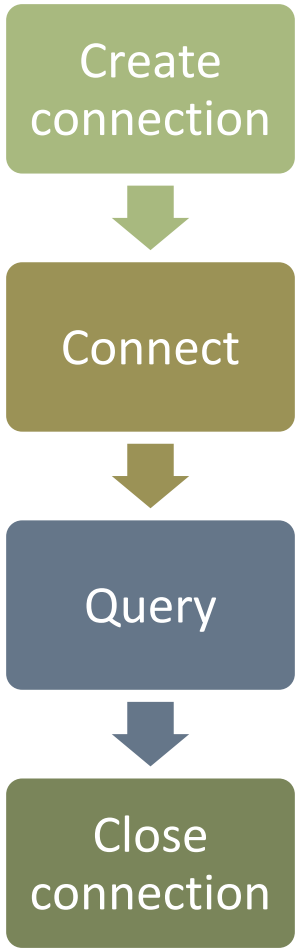
npm install mysql2



Simple query from database

select

myShopTest.js




```
const mysql = require('mysql2');
const con = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: '',
    database: 'myshop'
});
con.connect((err) => {
    if (err) throw err;
    console.log('Connected to the databas
    const sql = 'SELECT * FROM customers';
    con.query(sql, (err, results) => {
        if (err) throw err;
        console.log('Data received from Db:');
        console.log(results);
        con.end((err) => {
            if (err) throw err;
            console.log('Connection closed.');
```

```
Connected to the database.
Connected!
Data received from Db:
[
  { id: 1, name: 'Donald',
address: 'Black Street' },
  { id: 2, name: 'Sunny',
address: 'Amaly Old Town' }
]
Connection closed.
```

Insert into

```
con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
  let sql = "INSERT INTO customers (name, address) VALUES (?,?)";
  let values = ['Company Inc', 'Highway 37‘];
  con.query(sql, values, function (err, result) {
    if (err) throw err;
    console.log("1 record inserted");
  });
});
```

```
Connected!
1 record inserted
```

Insert into – multiple records

```
con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
  let sql = "INSERT INTO customers (name, address) VALUES ?";
  let values = [['John', 'Highway 71'], ['Peter', 'Lowstreet 4'], ['Amy', 'Apple st 652']];
  con.query(sql, [values], function (err, result) {
    if (err) throw err;
    console.log("Number of records inserted: " + result.affectedRows);
  });
});
```

```
Connected!
Number of records inserted: 3
```

The Result Object

a result object is returned from executing a query,

```
{
  fieldCount: 0,
  affectedRows: 3,
  insertId: 0,
  serverStatus: 2,
  warningCount: 0,
  message: '\'Records:4  Duplicated: 0  Warnings: 0',
  protocol41: true,
  changedRows: 0
}
```

Get Inserted ID

get the id of the row you just inserted to tables with an auto increment id field

result.insertId

```
let sql = "INSERT INTO customers (name, address) VALUES ('Michelle', 'Blue Village 1')";
con.query(sql, function (err, result) {
  if (err) throw err;
  console.log("1 record inserted, ID: " + result.insertId);
});
```

Update

```
con.connect(function(err) {
  if (err) throw err;
  let sql = "UPDATE customers SET address = ? WHERE address = ?";
  let values =  ['Canyon 123' , 'Valley 345‘];
  con.query(sql, values, function (err, result) {
    if (err) throw err;
    console.log(result.affectedRows + " record(s) updated");
  });
});
```

Delete

```
con.connect(function(err) {
  if (err) throw err;
  let sql = "DELETE FROM customers WHERE address = ?";
  let values = ['Mountain 21'];
  con.query(sql, values, function (err, result) {
    if (err) throw err;
    console.log("Number of records deleted: " + result.affectedRows);
  });
});
```

installation

npm init –y

npm install express

npm install mysql2







Node Express Database

```
const express = require('express')
const app = express()
const mysql = require('mysql2');
const port = 3000;
app.get('/customers', (req, res) => {
    const con = mysql.createConnection({
        host: 'localhost',
        user: 'root',
        password: '',
        database: 'myshop'
    });
    con.connect((err) => {
        if (err) console.log("connection err", err);
        const sql = 'SELECT * FROM customers';
        con.query(sql, (err, results) => {
            if (err) console.log("query err", err);
            res.send(results);
            con.end((err) => {
                if (err) console.log("end err", err);
            });
        });
```



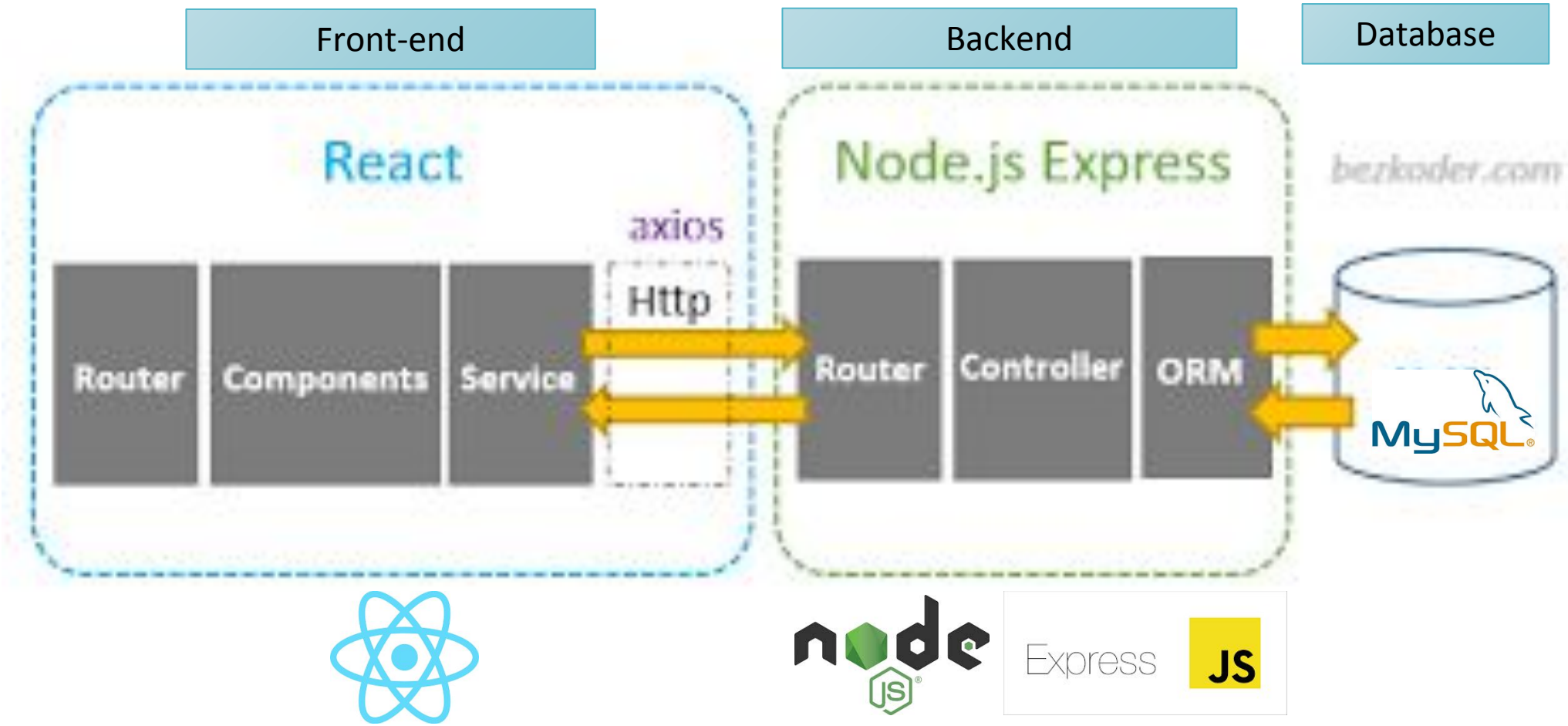
http://localhost:3000/customers


```
0:
  id:       1
  name:     "Donald"
  address:  "Black Street"
1:
  id:       2
  name:     "Sunny"
  address:  "Amaly Old Town"
2:
  id:       5
  name:     "Monae"
  address:  "Highway 37"
3:
  id:       6
  name:     "John"
  address:  "Highway 71"
```



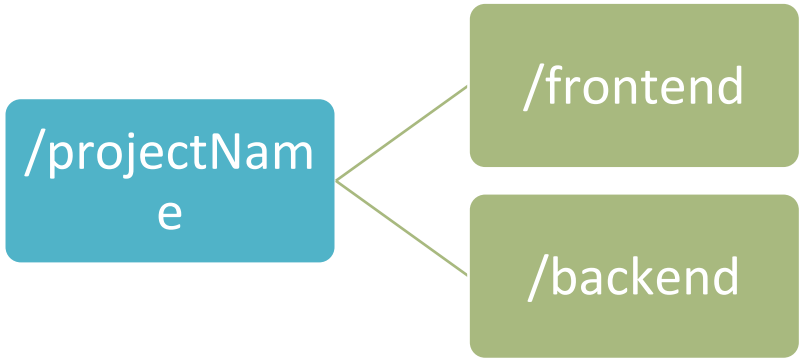
Full Stack

Architecture



https://www.bezkoder.com/react-node-express-mysql/

Create project

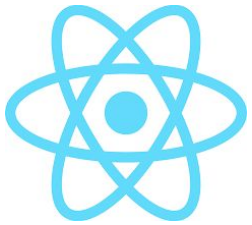


FRONT-END

/frontend

Install ReactJs inside frontend

Install dependencies



AXIOS

npm install [dependency]

BACK-END

/backend

>npm init –y

Install dependencies

Express JS



cores

.env

database

MySQL2

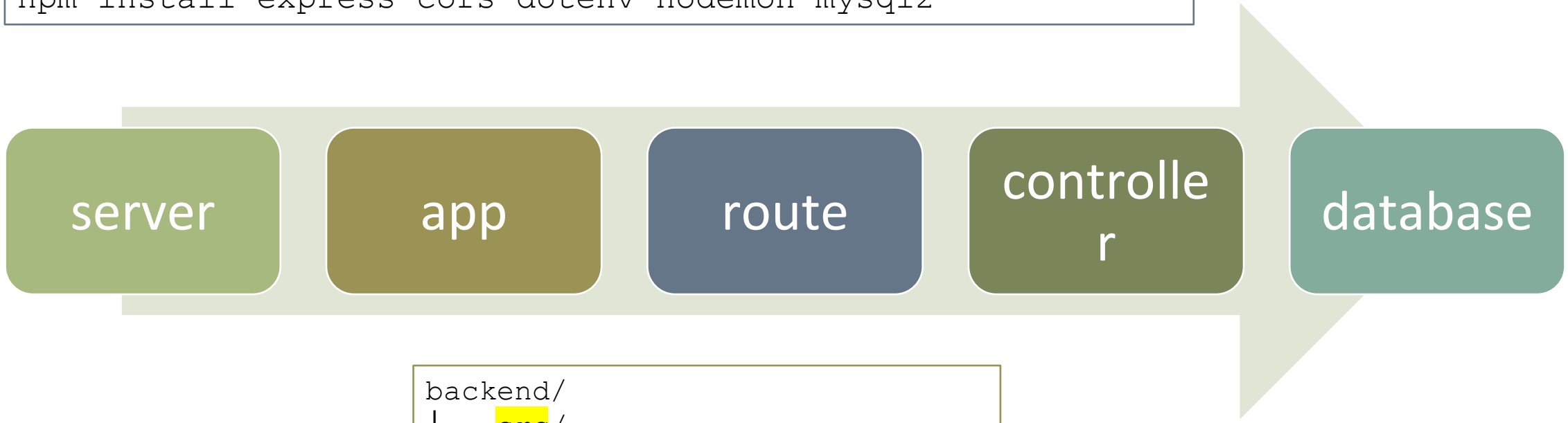
Prisma

Cores -> Cross-Origin Resource Sharing



Backend

```
npm init -y
npm install express cors dotenv nodemon mysql2
```




```
backend/
├── src/
│   ├── controllers/
│   │   └── userController.js
│   ├── db/
│   │   └── pool.js
│   ├── routes/
│   │   └── userRoutes.js
│   ├── app.js
│   └── server.js
├── .env
└── package.json
```

backend/.env

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=myshop

PORT=5000


DATABASE_URL="mysql://root:@localhost:3306/myshop"
```

backend/db/pool.js

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
              waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0
});

module.exports = pool;
```



Async / Await

API

Get all	GET	/api/users
Get id	GET	/api/users/:id
Search	GET	/api/users/search?keyword=key
Insert	POST	/api/users
Update	PUT	/api/users/:id
Delete	DELETE	/api/users/:id

backend/src/controllers/userController.js

```
const pool = require('../db/pool');

const getUsers = async (req, res) => {
    try {
        const [rows] = await pool.query('SELECT * FROM users');
        res.json(rows);
    } catch (err) {
        console.error('Error fetching users:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

getById

```
const getUserById = async (req, res) => {
    const { id } = req.params;
    try {
        const [rows] = await pool.query('SELECT * FROM users WHERE id = ?', [id]);
        if (rows.length === 0) {
            return res.status(404).json({ error: 'User not found' });
        }
        res.json(rows[0]);
    } catch (err) {
        console.error('Error fetching user:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

Search

```
const searchUsers = async (req, res) => {
    const { keyword } = req.query;
    if (!keyword) {
        return res.status(400).json({ error: 'Please provide a search keyword' });
    }
    try {
        const searchTerm = `%${keyword}%`;
        const [rows] = await pool.query(
            'SELECT * FROM users WHERE name LIKE ? OR email LIKE ?',
            [searchTerm, searchTerm]
        );
        res.json(rows);
    } catch (err) {
        console.error('Error searching users:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

create

```
const createUser = async (req, res) => {
    const { name, email } = req.body;
    if (!name || !email) {
        return res.status(400).json({ error: 'Name and email are required' });
    }
    try {
        const [result] = await pool.query(
            'INSERT INTO users (name, email) VALUES (?, ?)',
            [name, email]
        );
        res.status(201).json({ id: result.insertId, name, email });
    } catch (err) {
        console.error('Error creating user:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

update

```
const updateUser = async (req, res) => {
    const { id } = req.params;
    const { name, email } = req.body;
    if (!name || !email) {
        return res.status(400).json({ error: 'Name and email are required' });
    }
    try {
        const [result] = await pool.query(
            'UPDATE users SET name = ?, email = ? WHERE id = ?',
            [name, email, id]
        );
        if (result.affectedRows === 0) {
            return res.status(404).json({ error: 'User not found' });
        }
        res.json({ id: Number(id), name, email });
    } catch (err) {
        console.error('Error updating user:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

delete

```
const deleteUser = async (req, res) => {
    const { id } = req.params;
    try {
        const [result] = await pool.query('DELETE FROM users WHERE id = ?',
[id]);
        if (result.affectedRows === 0) {
            return res.status(404).json({ error: 'User not found' });
        }
        res.json({ message: 'User deleted successfully', id: Number(id) });
    } catch (err) {
        console.error('Error deleting user:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

```
module.exports = {
  getUsers,
  getUserById,
  searchUsers,
  createUser,
  updateUser,
  deleteUser
};
```

backend/src/routes/userRoutes.js

```
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');

// Route search (put before  /:id)
router.get('/search', userController.searchUsers);   // search

// Routes CRUD
router.get('/', userController.getUsers);              // getAll
router.get('/:id', userController.getUserById);      // getById
router.post('/', userController.createUser);          // Create
router.put('/:id', userController.updateUser);        // Update
router.delete('/:id', userController.deleteUser);    // Delete

module.exports = router;
```

backend/src/app.js

```
const express = require('express');
const cors = require('cors');
const userRoutes = require('../routes/userRoutes');
const app = express();
// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
// Routes Mounting
app.use('/api/users', userRoutes);
// Root endpoint
app.get('/', (req, res) => {
    res.send('API is running...');
});
module.exports = app;
```

backend/src/server.js

```
const app = require('./app');
require('dotenv').config();

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
    console.log(`Server is running on port ${PORT}`);
});
```

backend/package.json

```
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
```

npm run dev

API Check

GetAll	GET /api/users
GetById	GET /api/users/:id
Insert	POST /api/users
Update	PUT /api/users/:id
Delete	DELETE /api/users/:id
Search	GET /api/users/search?keyword=key

Insert

- **URL:** http://localhost:5000/api/users
- **Method:** POST
- **Headers:** Content-Type: application/json
- **Body (Raw JSON):** json
  - { "name": "John Doe", "email": "john@example.com"}

Update

- **URL:** http://localhost:5000/api/users/1
- **Method:** PUT
- **Headers:** Content-Type: application/json
- **Body (Raw JSON):** json
  - { "name": "John Updated", "email": "john.updated@example.com"}

Test result with

Postman

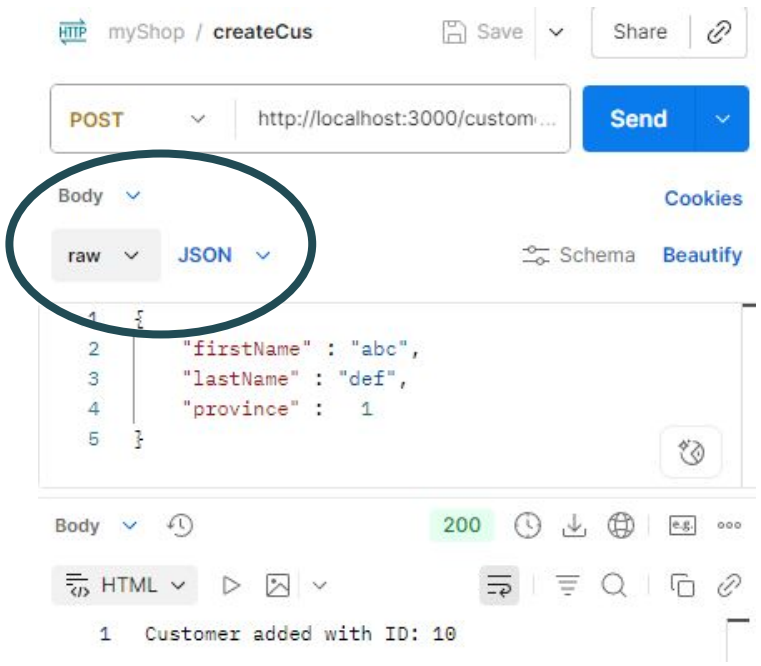
express.json is middleware parses incoming requests with the JSON string into a JavaScript object

Express.urlencoded parses incoming requests with URL-encoded payloads, typically from HTML form submissions

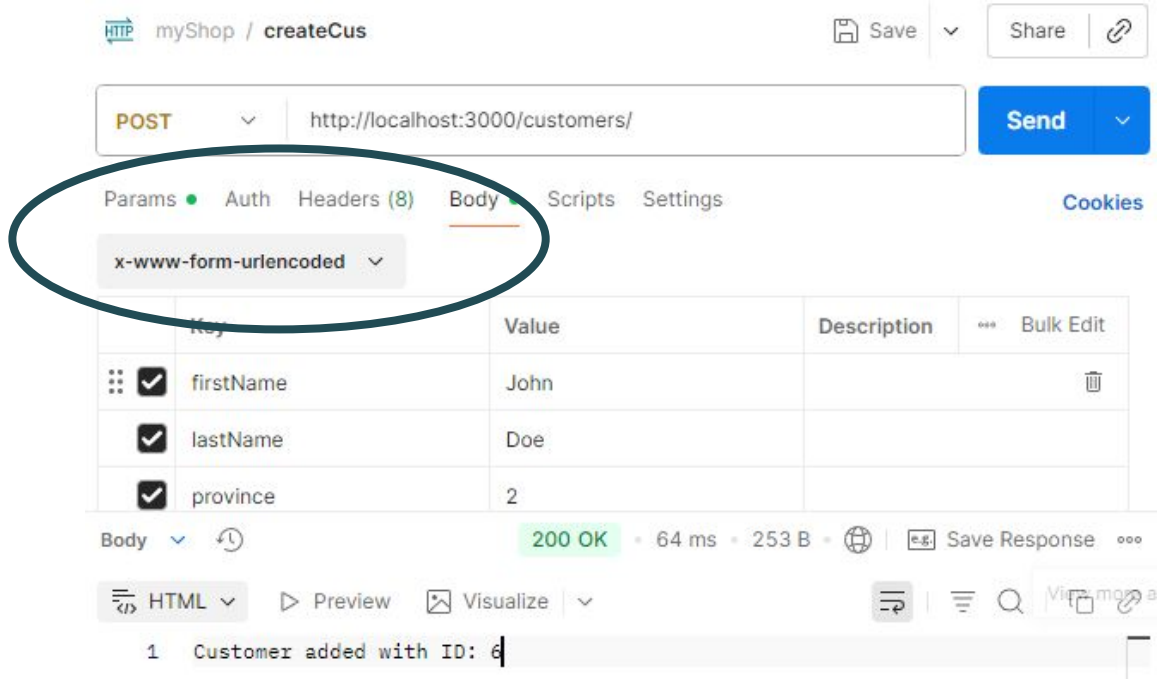



```
const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
```

raw JSON



urlencoded



npm install body-parser

Test result with cmd

GetAll	curl http://localhost:5000/api/users
GetById	curl http://localhost:5000/api/users/1
Insert	curl -X POST http://localhost:5000/api/users -H "Content-Type: application/json" -d "{\"name\":\"Somchai Jaidee\",\"email\":\"somchai@example.com\"}"
Update	curl -X PUT http://localhost:5000/api/users/1 -H "Content-Type: application/json" -d "{\"name\":\"Somchai Updated\",\"email\":\"new@example.com\"}"
Delete	curl -X DELETE http://localhost:5000/api/users/1
Search	curl http://localhost:5000/api/users/search?keyword=Somchai

PRISMA

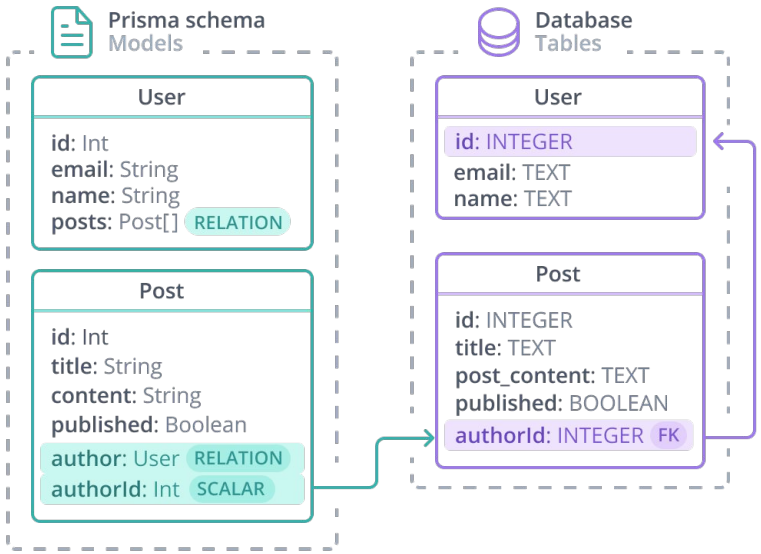
PRISMA

Prisma is an open-source ORM for Node.js and TypeScript.

It is used as an alternative to writing plain SQL, or using another database access tool

ORMs provide interface through objects to create, read, delete, and manipulate data while hiding some of the complexity of the database





https://www.prisma.io/

Installation

>npm install express cores dotenv nodemon mysql2

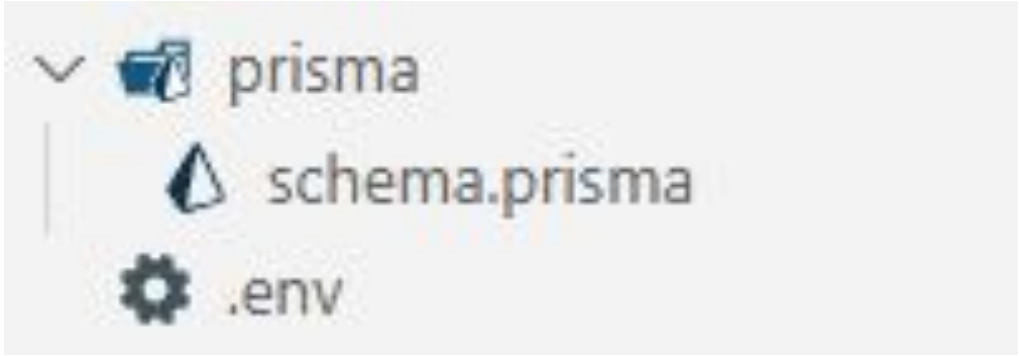
>npm install prisma

>npm install @prisma/client

>npm install @prisma/adapter-mariadb

>npx prisma init





https://www.prisma.io/docs/orm

Database Setting

Set provider – mysql

/prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}
datasource db {
  provider = "mysql"
}
```

Remove output = "../generated/prisma"

Connect to mysql external

Database Connection

.env

```
DATABASE_URL="mysql://USER:PASSWORD@HOST:PORT/DATABASE"
```

Pull database

Map database

```
npx prisma db pull
```



/prisma/schema.prisma

```
model customers {
  customerId Int       @id @default(autoincrement())
  firstName  String    @db.VarChar(40)
  lastName   String    @db.VarChar(40)
  provinceId Int
  provinces  provinces @relation(fields: [provinceId],

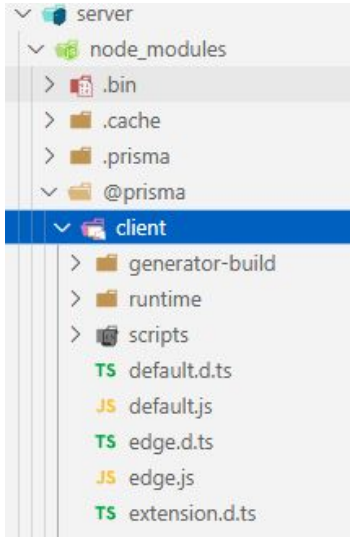
  @@index([provinceId], map: "province")
}

model provinces {
  provinceId Int         @id @default(autoincrement())
  name       String      @db.VarChar(50)
  customers  customers[]
}
```

Generate Prisma Client to interact with database

```
npx prisma generate
```





Create prisma module (dbpool)

db/prisma.js

```
const { PrismaClient } = require('@prisma/client');
const { PrismaMariaDb } = require('@prisma/adapter-mariadb');
require('dotenv').config();

const mariadb = require('mariadb');

const pool = mariadb.createPool({
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
    connectionLimit: 5
});
const adapter = new PrismaMariaDb(pool);
const prisma = new PrismaClient({ adapter });

module.exports = prisma;
```

Simple server

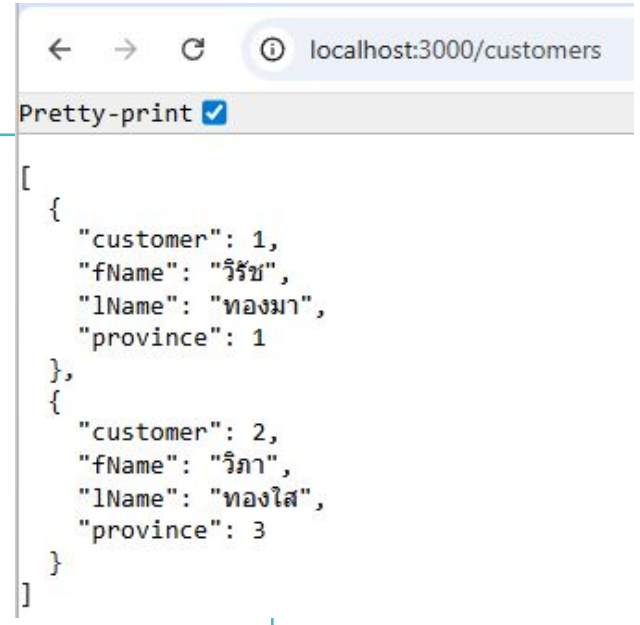
prismaTest.js

```
const express = require('express');
const prisma = require('./db/prisma');
const app = express();
const port = 3000;

//middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.get('/customers', async (req, res) => {
    const customers = await prisma.customers.findMany();
    res.send(customers)
});

app.listen(port, () => { console.log("server start at port ", port)
})
```



Send error with try/ catch

```
app.get('/customers', async (req, res) => {
    try {
        const customers = await prisma.customers.findMany();
        res.send(customers);



    } catch (err) {
        console.log(err)
        res.send("error").status(500);
    }
});
```

getAll

Controller

controllers/customerController.j
s

```
const prisma = require('../db/prisma');

const getCustomers = async (req, res) => {
    try {
        let customers = await prisma.customers.findMany();
        res.json(customers);
    } catch (err) {
        console.error('Error fetching customers:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

```
const getCustomerById = async (req, res) => {
    const { id } = req.params;
    try {
        const customer = await prisma.customers.findUnique({
            where: { id: Number(id) }
        });
        if (!customer) {
            return res.status(404).json({ error: 'Customer not found' });
        }
        res.json(customer);
    } catch (err) {
        console.error('Error fetching customer:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

Create

```
const createCustomer = async (req, res) => {
    let { fName, lName, province } = req.body;
    if (!fName || !lName || !province) {
        return res.status(400).json({ error: 'Name and province are required' });
    }
    try {
        const newCustomer = await prisma.customers.create({
            data: {
                fName: fName,
                lName: lName,
                province: Number(province)
            }
        });
        res.status(201).json(newCustomer);
    } catch (err) {
        console.error('Error creating customer:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

```
const updateCustomer = async (req, res) => {
    const { id } = req.params;
    const { fName, lName,province } = req.body;
    try {
        const updatedCustomer = await prisma.customers.update({
            where: { id: Number(id) },
            data: {
                fName : fName,
                lName : lName,
                province : Number(province)
            }
        });
        res.json(updatedCustomer);
    } catch (err) {
        console.error('Error updating customer:', err);
        res.status(404).json({ error: 'Customer not found or database error' });
    }
};
```

Delete

```
const deleteCustomer = async (req, res) => {
    const { id } = req.params;
    try {
        await prisma.customers.delete({
            where: { id: Number(id) }
        });
        res.json({ message: 'Customer deleted successfully', id: Number(id) });
    } catch (err) {
        console.error('Error deleting customer:', err);
        res.status(404).json({ error: 'Customer not found' });
    }
};
```

```
module.exports = {
    getCustomers,
    getCustomerById,
    createCustomer,
    updateCustomer,
    deleteCustomer
};
```

Search

```
const getCustomers = async (req, res) => {
    const { keyword } = req.query;
    try {
        let customers;
        if (keyword) {
            customers = await prisma.customers.findMany({
                where: {
                    OR: [
                        { fName: { contains: keyword } },
                        { lName: { contains: keyword } }
                    ]
                }
            });
        } else {
            customers = await prisma.customers.findMany();
        }
        res.json(customers);
    } catch (err) {
        console.error('Error fetching customers:', err);
        res.status(500).json({ error: 'Database error' });
    }
};
```

/api/customers/?keyword=aaa

/api/customers

Routes

routes/customerRoutes.j

```
const express = require('express');
const router = express.Router();
const customerController =
require('../controllers/customerController');

router.get('/', customerController.getCustomers);
router.get('/:id', customerController.getCustomerById);
router.post('/', customerController.createCustomer);
router.put('/:id', customerController.updateCustomer);
router.delete('/:id', customerController.deleteCustomer);

module.exports = router;
```

app.js

```
const customerRoutes = require('./routes/customerRoutes');

app.use('/api/customers', customerRoutes);
```

```
const customers = await prisma.customers.findMany({
            include:{
                provinces:true
            }
        });
```

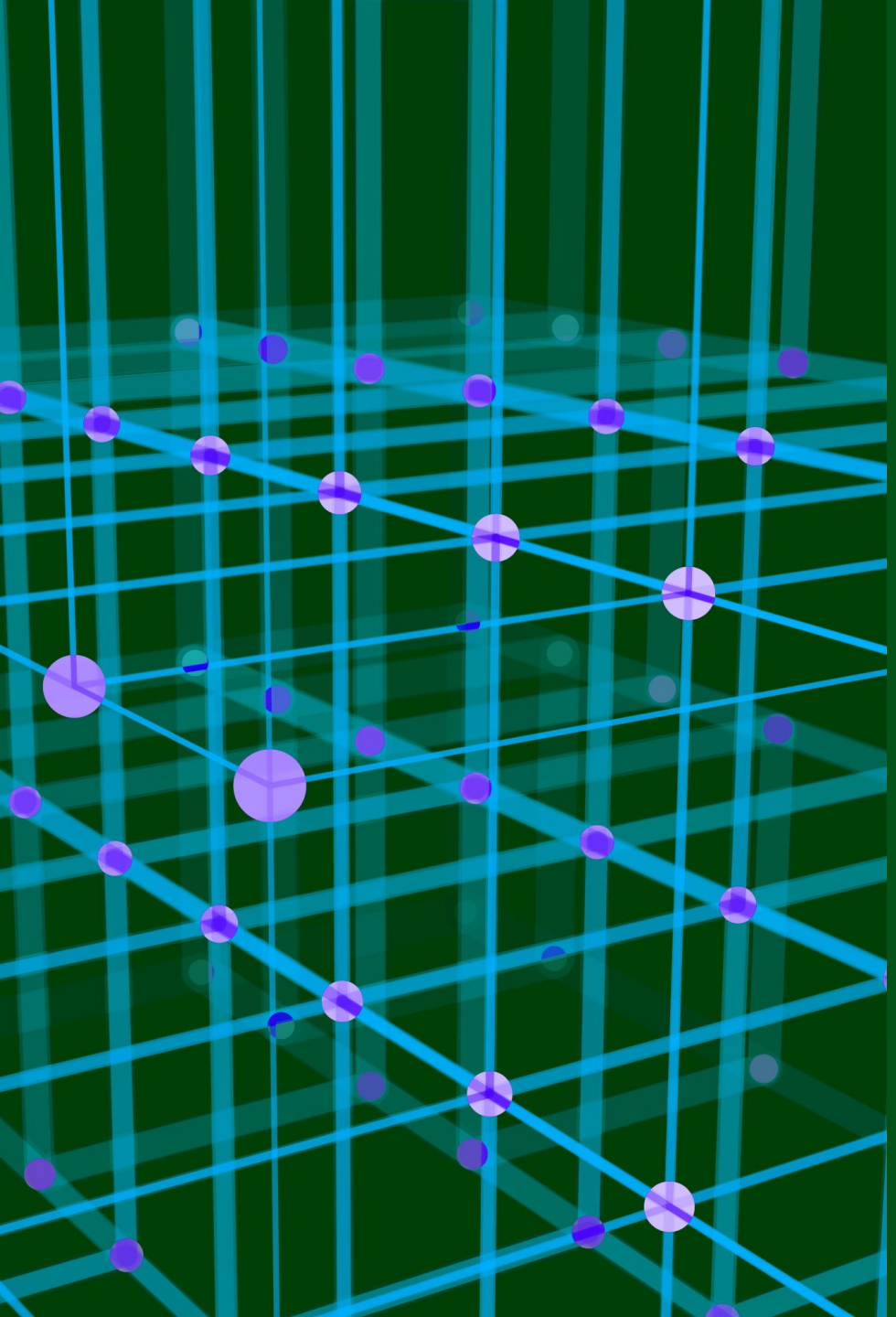


```
{
  "customerId": 1,
  "firstName": "วิรัช",
  "lastName": "ทองมา",
  "provinceId": 1,
  "provinces": {
    "provinceId": 1,
    "name": "กรุงเทพ"
  }
},
```

Check relationship field name from schema.prisma

```
model customers {
  id
  fName
  lName
  province
  provinces   province @relation("custo

  @@index([province], map: "province")
}
```



Todo

1. สร้าง API สำหรับ menu และชนิด

/api/menu

/api/type

